

Security Audit

Metaversal (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	24
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Metaversal team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Metaversal.gg is a next-generation metaverse platform that fuses immersive virtual experiences with decentralized finance and community governance. It offers a dynamic ecosystem where users can explore, build, and trade digital assets across interconnected virtual worlds. Leveraging blockchain technology, NFTs, and smart contracts, Metaversal.gg empowers creators and players to monetize their experiences while participating in decentralized decision-making. Step into the future of digital interaction, where every virtual space is a gateway to endless creativity, robust financial opportunities, and a truly interoperable metaverse powered by Web3 innovation.

Project Name: Metaversal

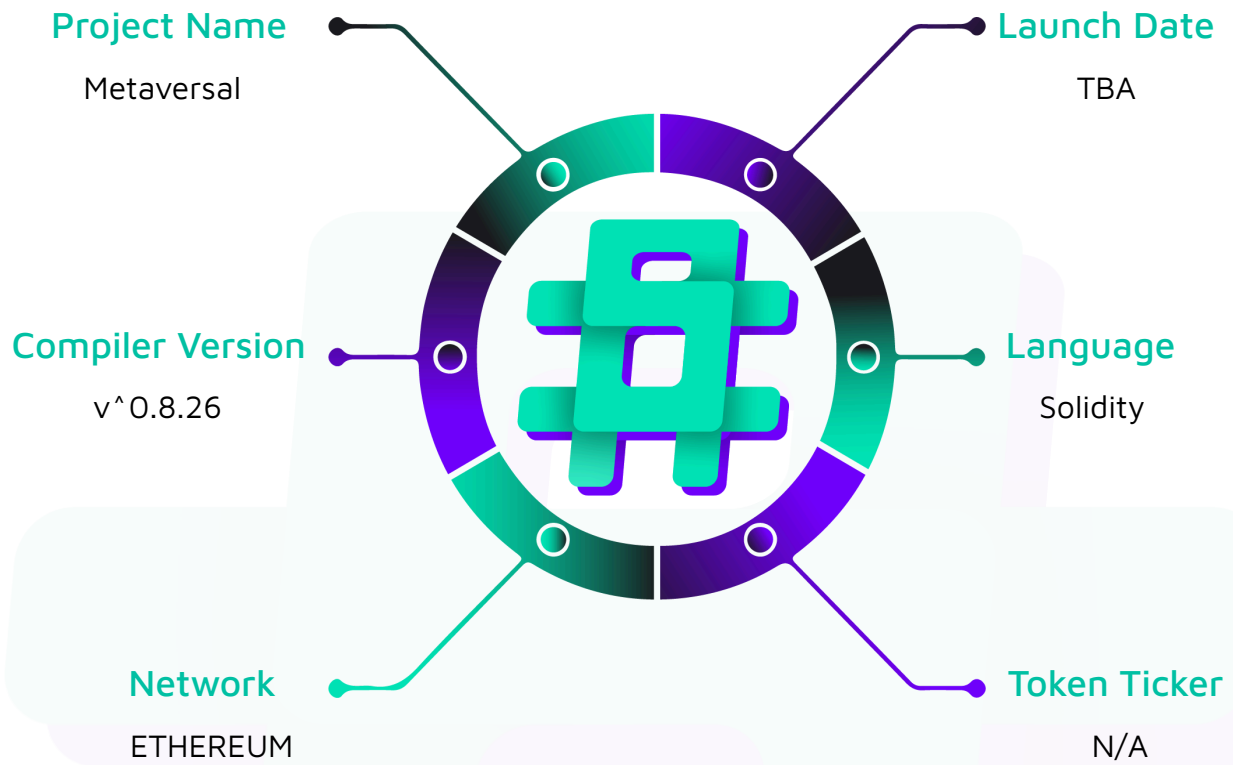
Project Type: Defi

Compiler Version: ^0.8.26

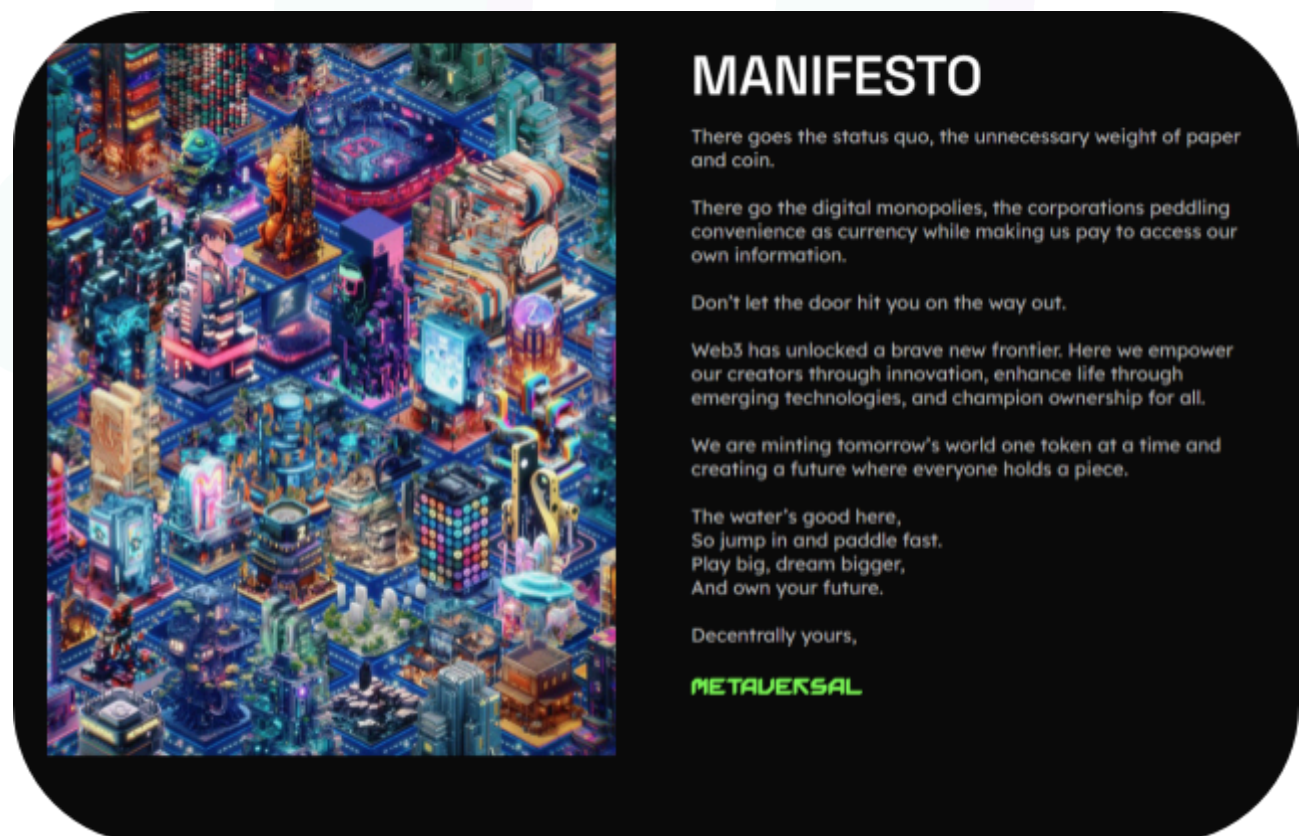
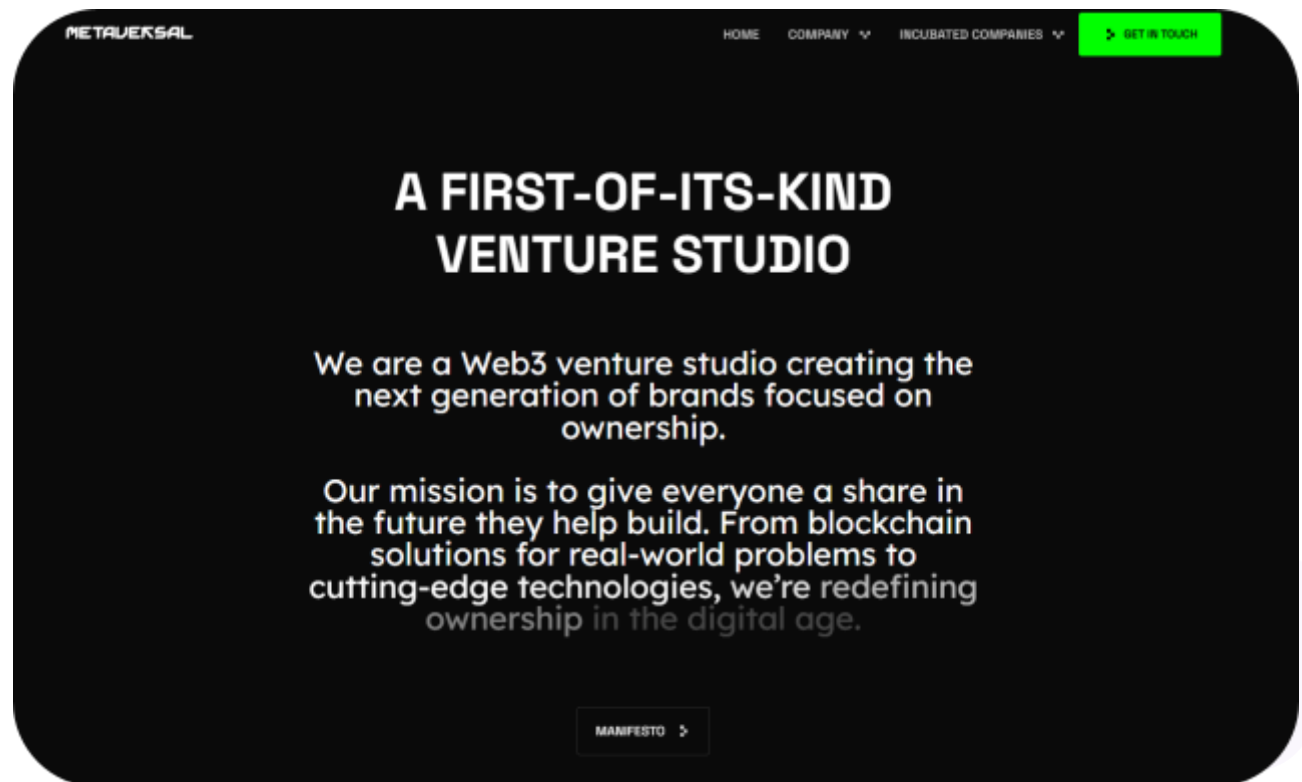
Website: <http://metaversal.gg/>

Logo:

The logo for Metaversal, featuring the word "METAVERSAL" in a bright green, pixelated, blocky font. The text is centered within a solid black rounded rectangular background.

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Metaversal project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Metaversal Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	March, 2025
Contract 1	EthLiquidityPool.sol
Contract 1 MD5 Hash	13fc7f076479203e605143033f79f53c
Contract 2	LiquidityPoolBase.sol
Contract 2 MD5 Hash	b009740e7fedb464d0a3182557ce074b
Contract 3	UsdcLiquidityPool.sol
Contract 3 MD5 Hash	55c0b89a7b7aebb1675817f95d6e24a8
Contract 4	MockERC20.sol
Contract 4 MD5 Hash	00d7ef0c181b134d5ff93edfcdb00b91
Contract 5	ReentrancyAttacker.sol
Contract 5 MD5 Hash	50a115134ecfb8e3131f27e74693280d
Contract 6	TokenReentrancyAttacker.sol
Contract 6 MD5 Hash	c0b8c37954c3bab45061f2f0a8490275
GitHub Commit Hash	942ef37c1fcafe5649282b027f30dca09d961421

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 Medium severity vulnerabilities

5 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
LiquidityPoolBase.sol Allows users to: - Deposit funds for a fixed duration to earn interest - Withdraw funds and accrued interest once lock period expires - View details of their deposits including interest earned and time remaining Allows admins to: - Configure pool settings (cap, duration, APY) - Manage depositors (blacklist, force returns) - Control fund flows (withdraw, pause operations) - Execute multi-step pool liquidation process	Contract achieves this functionality.
EthLiquidityPool.sol Allows users to: - Deposit ETH into the pool to earn interest - Withdraw their ETH and accrued interest once lock period expires Allows admins to: - Deposit additional ETH into the pool - Access all inherited functionality from LiquidityPoolBase	Contract achieves this functionality.
UsdcLiquidityPool.sol Allows users to: - Deposit ETH into the pool to earn interest	Contract achieves this functionality.

- Withdraw their ETH and accrued interest once the lockperiod expires Allows admins to: - Deposit additional ETH into the pool - Access all inherited functionality from LiquidityPoolBase	
MockERC20.sol - A simple test-only contract: - Creates mock ERC20 tokens with custom decimals - Allows anyone to mint unlimited tokens for testing	Contract achieves this functionality.
ReentrancyAttacker.sol A testing contract that: - Simulates various reentrancy attacks on EthLiquidityPool - Attempts to exploit withdrawal functions through callbacks	Contract achieves this functionality.
TokenReentrancyAttacker.sol -A test contract that simulates ERC20-specific reentrancy attacks against UsdcLiquidityPool to verify security protections.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Metaversal project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Metaversal project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] UsdcLiquidityPool#withdraw - Depositors can lose all their deposited tokens if their address is blacklisted

Description

The withdraw function in UsdcLiquidityPool sends tokens to msg.sender without allowing the caller to specify a withdrawal destination address. If a user's address is blacklisted by USDC after depositing, their funds will become permanently locked in the contract with no recovery method.t.

Vulnerability Details

The current implementation of the withdraw function in UsdcLiquidityPool sends tokens to the caller's address::

```
function withdraw(uint256 depositId) external whenWithdrawalsNotPaused {  
    _withdraw(depositId);  
}
```

The underlying _withdraw function transfers tokens to _msgSender() without providing any option to specify an alternative recipient. USDC has an active blacklisting mechanism that can prevent transfers to specific addresses. If a user becomes blacklisted after depositing funds, any attempt to withdraw will fail as the USDC transfer operation will revert. Since there's no way to specify an alternative withdrawal address, the user's funds become permanently locked in the contract..

Impact

This vulnerability can lead to permanent loss of user funds if they become blacklisted by USDC after making deposits.

Recommendation

Modify the withdraw function to accept an optional recipient address parameter

Status

Resolved

[M-02] LiquidityPoolBase#toggleWithdrawals - Owner Can Block Withdrawals After Liquidation, Preventing Users From Accessing Their Funds

Description

The `toggleWithdrawals` function allows the owner to pause withdrawals even after liquidation is finalized. This contradicts the core purpose of the liquidation process, which is meant to enable users to withdraw their proportional share of funds. While deposits are properly locked after liquidation, withdrawals can still be paused by the owner

Vulnerability Details

The `toggleWithdrawals` function allows the owner to enable or disable withdrawal functionality in the liquidity pool.

```
function toggleWithdrawals(bool paused) external onlyOwner {  
  
    require(!liquidated || paused, "Cannot unpause withdrawals after liquidation");  
  
    withdrawalsPaused = paused;  
  
    emit WithdrawalsToggled(paused);  
  
}
```

Here this function allows the pool owner to permanently block user withdrawals after liquidation. While the condition `require(!liquidated || paused, "Cannot unpause withdrawals after liquidation")` intends to restrict functionality post-liquidation, it actually permits the owner to pause withdrawals even after liquidation is finalized. This creates a denial-of-service scenario where users cannot access their funds after

liquidation, contradicting the core purpose of the liquidation process which explicitly enables withdrawals in the `finalizeLiquidation` function.

Impact

This issue enables the owner to completely block users from accessing their funds after liquidation, effectively breaking the core functionality of the liquidation process.

Recommendation

Modify the `toggleWithdrawals` function to completely prevent withdrawal pausing after liquidation.

Status

Resolved

[M-03] LiquidityPoolBase#_makeDeposit - Unbounded State Growth Due to Lack of Minimum Deposit Threshold

Description

The absence of a minimum deposit amount check in the `_makeDeposit` function allows attackers to create numerous tiny deposits, permanently bloating the contract state and potentially disrupting the liquidation process.

Vulnerability Details

The `_makeDeposit` function is the core internal function that handles user deposits into the liquidity pool

```
function _makeDeposit(address lender, uint256 amount) internal {  
    require(!blacklisted[lender], "Address is blacklisted");  
    require(amount > 0, "Amount must be greater than 0");  
    require(totalDeposited + amount <= poolCap, "Deposit exceeds pool cap");  
    require(!liquidated, "Pool is liquidated");  
}
```

```

// Track new depositors

if (!isDepositor[lender]) {

    isDepositor[lender] = true;

    depositors.push(lender);

}

deposits[lender].push(Deposit({amount: amount, depositTime: block.timestamp,
duration: lockDuration, apyBasisPoints: apy, withdrawn: false}));

totalDeposited += amount;

emit DepositMade(lender, amount, deposits[lender].length - 1);
}

```

This function only checks that the deposit amount is greater than zero (`require(amount > 0)`), allowing deposits as small as 1 wei. Each new depositor address is permanently added to the `depositors` array with no mechanism to remove them even after withdrawals. During liquidation, the protocol processes all depositors in batches, iterating through each depositor and their deposits. Without a reasonable minimum threshold, an attacker could make many minimal deposits from different addresses, causing permanent state bloat.

Impact

This vulnerability enables a low-cost griefing attack that permanently increases gas costs for contract operations, especially during liquidation. In extreme cases, an attacker could force the liquidation process to exceed block gas limits by adding thousands of distinct depositor addresses.

Recommendation

Implement a reasonable minimum deposit threshold. Additionally, consider adding logic to remove users from the `depositors` array after they've withdrawn all their deposits

Status

Resolved

[M-04] LiquidityPoolBase#_withdraw - Unprotected Withdrawals Before Liquidation Enable Front-running

Description

Users whose lock periods have expired can frontrun the liquidation process by withdrawing their funds at full value, potentially leaving insufficient funds for remaining depositors.

Vulnerability Details

When a deposit's lock period expires, users can withdraw their full principal plus accrued interest. Meanwhile, if the pool's actual balance is below 100% of the required funds (principal + interest for all depositors), it would normally trigger liquidation where all depositors receive a proportional share based on the distribution ratio. However, there is no synchronization mechanism between these two states. Users monitoring the blockchain can detect when the pool admin is preparing to liquidate and frontrun this transaction by withdrawing their full funds first. The code specifically allows this as there's no liquidity check during withdrawals:

```
// Regular withdrawal path (no liquidation)

uint256 amount = userDeposit.amount;

uint256 interest = calculateInterest(userDeposit);

uint256 totalAmount = amount + interest;
```

This creates a "first to withdraw, first to be made whole" situation, unfairly benefiting users who withdraw quickly at the expense of those who remain.

Impact

This vulnerability creates a race condition that disadvantages later withdrawers, violating the principle of fair fund distribution during financial distress. Early withdrawers receive 100% of their funds while later withdrawers may receive



substantially less after liquidation is activated. In extreme cases, if many depositors frontrun liquidation, the remaining depositors might receive a severely reduced share

Recommendation

Add a function to temporarily freeze all withdrawals when the pool balance falls below a certain threshold, allowing time for a proper liquidation process.

Note

The Metaversal team informed Hashlocked that the behavior identified in M-04 is an intentional design choice that allows the pool balance to temporarily drop below the total deposits, hence the team is not going to alter the code. This decision was made to keep the system flexible and allow users to withdraw their full funds when their lock period expires, even if it means that the overall balance might briefly be lower than expected before funds are rebalanced with interest. In other words, the team accepts a short window where frontrunning could occur as a trade-off for maintaining immediate liquidity for users, and they have built the protocol so that any temporary imbalance is automatically corrected later.

Status

Acknowledged

Low

[L-01] Contracts - Use of floating pragma

Description

The contracts `EthLiquidityPool.sol`, `LiquidityPoolBase.sol`, and `UsdcLiquidityPool.sol` use a floating pragma version (^0.8.26). It's crucial to compile and deploy contracts with the same compiler version and settings used during development and testing. Fixing the pragma version prevents compilation with different versions. Using an outdated pragma version could introduce bugs that negatively affect the contract system, while newly released versions may have undiscovered security vulnerabilities.

Recommendation

Change the pragma statement in both contracts to a fixed version, such as `pragma solidity 0.8.26`. This locks the contract to a specific compiler version.

Status

Resolved

[L-02] Contracts - Missing Input Validation in LiquidityPoolBase Constructor

Description

The `LiquidityPoolBase` contract's constructor does not validate any of its input parameters (`initialOwner`, `initialPoolCap`, `initialLockDuration`, `initialApy`). This lack of validation could allow the contract to be deployed in an inconsistent or invalid state.

```
constructor(address initialOwner, uint256 initialPoolCap, uint256 initialLockDuration,
uint256 initialApy) Ownable(initialOwner) {

    poolCap = initialPoolCap;

    lockDuration = initialLockDuration;

    apy = initialApy;

}
```

Recommendation

Add input validation checks in the constructor to ensure all parameters are within reasonable and expected ranges

Status

Resolved

[L-03] Contracts - Inconsistent Dust Collection Between User Withdrawals and Admin Force Returns

Description

The contract implements dust collection logic in the regular withdrawal function but omits it in the admin-forced withdrawal function (`forceReturn`). When a user withdraws through the `_withdraw` function, the contract checks if it's the final withdrawal and distributes any remaining dust amounts to that user. However, this logic is missing from the `forceReturn` function. If an admin uses `forceReturn` for the final withdrawal from the pool, any dust amounts will be trapped in the contract.

```
function forceReturn(address lender, uint256 depositId) external onlyOwner nonReentrant
validDepositId(lender, depositId) {

    Deposit storage userDeposit = deposits[lender][depositId];

    require(!userDeposit.withdrawn, "Deposit already withdrawn");

    uint256 amount = userDeposit.amount;

    uint256 interest = calculateInterest(userDeposit);

    uint256 totalAmount = amount + interest;

    // Apply distribution ratio if liquidated

    if (liquidated) {

        // Calculate user's share of the total claims

        uint256 userShare = (totalAmount * distributionRatio) / 10000;
```

```

        // Make sure we're distributing the correct proportion

        totalAmount = userShare;

    }

    userDeposit.withdrawn = true;

    totalDeposited -= amount;

    _transferToUser(lender, totalAmount);

    emit ForcedReturn(lender, amount, interest, depositId);

}

```

Recommendation

Add the dust collection logic to the `forceReturn` function to ensure consistency with the regular withdrawal process.

Status

Resolved

[L-04] Contracts - Misleading Event Emission in `startLiquidation` Function

Description

The `startLiquidation` function in the `LiquidityPoolBase` contract emits a `LiquidationBatchProcessed` event with all zero values when the liquidation process is initiated. This can be misleading as the event name suggests that a batch of depositors has been processed, whereas the function is merely starting the liquidation process. This could lead to confusion for developers and users who rely on event logs to track the state and progress of the contract.

Recommendation

Introduce a new event named `LiquidationStarted` to clearly indicate the initiation of the liquidation process.

Status

Resolved

[L-05] Contracts- Missing Input Validation in setPoolCap and setTerms Functions

Description

The LiquidityPoolBase contract's setPoolCap and setTerms functions lack proper input validation for their parameters. This absence of validation could allow the contract owner to set critical pool parameters to unreasonable or potentially harmful values.

```
function setPoolCap(uint256 newCap) external onlyOwner {  
    require(!liquidated, "Pool is liquidated");  
    poolCap = newCap;  
    emit PoolCapUpdated(newCap);  
}  
  
function setTerms(uint256 newDuration, uint256 newApy) external onlyOwner {  
    require(!liquidated, "Pool is liquidated");  
    lockDuration = newDuration;  
    apy = newApy;  
    emit TermsUpdated(newDuration, newApy);  
}
```

Recommendation

Add input validation checks to both functions to ensure parameters are within reasonable and expected ranges.

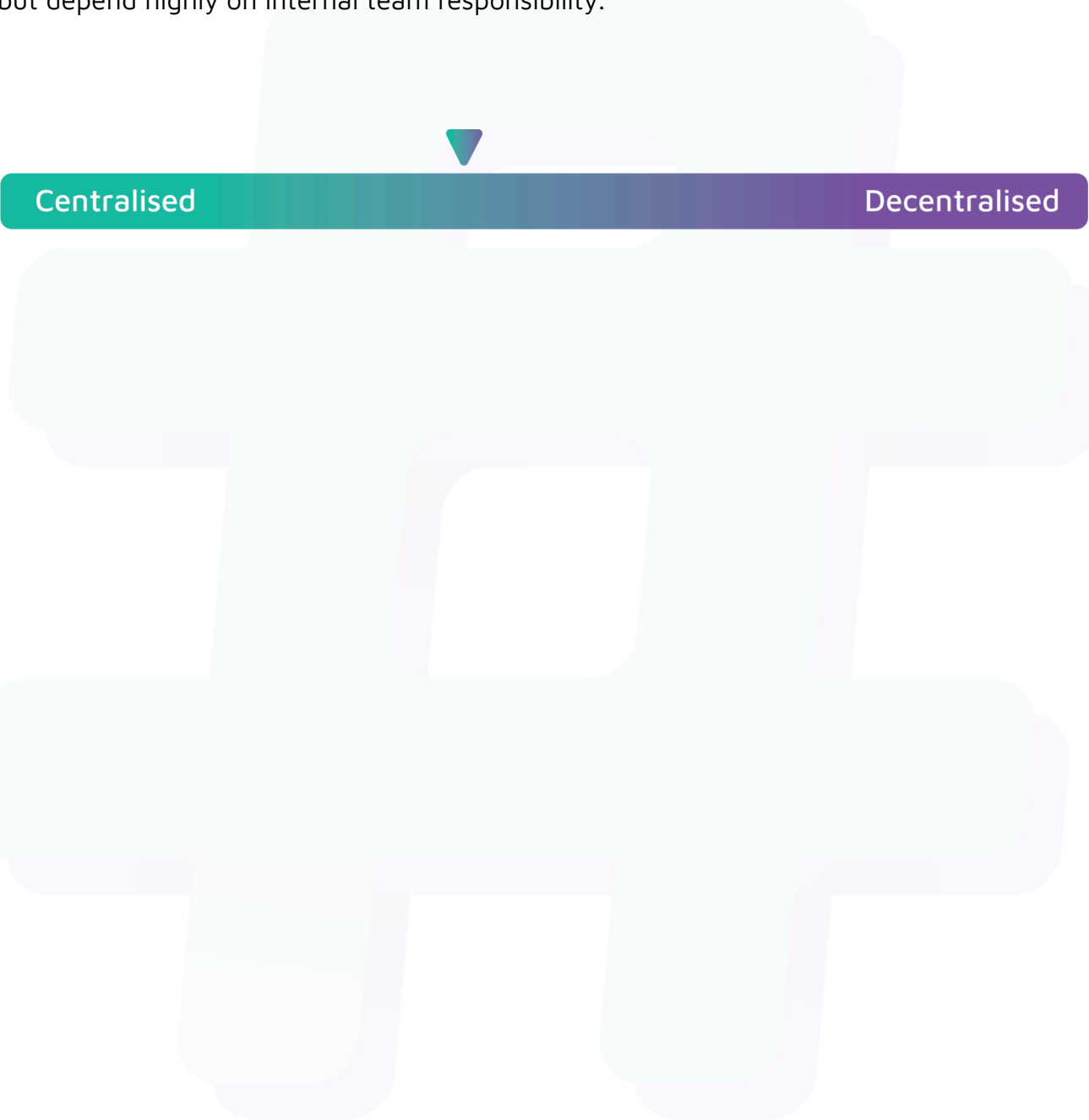
Status

Resolved

Centralisation

The Metaversal project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Metaversal project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.